Politecnico di Torino

Cybersecurity for Embedded Systems

Trusted Platform Module (TPM) Implementation

Project Documentation

Master Degree in Computer Engineering

Referents: Prof. Alessandro Savino, Nicola Scarano

Authors:

Maganuco Giuseppe - s345962
Bonenti Silvia - s339232
Marocco Luca - s333945

July 15, 2025

Contents

CHAPTER 1

Introduction

1.1 Hardware-based Security

This chapter provides an introduction to the Hardware-based Security components, which are essential to build trust at the physical level of a system; they protect against attacks that can't be mitigated by software alone.

1.1.1 Hardware Trust

Hardware trust is a critical aspect of modern computing systems, ensuring that the hardware components are genuine and have not been tampered with.

This trust is established through various mechanisms, including secure boot processes, hardware root of trust, and the use of trusted platform modules (TPMs).

1.1.2 Root of Trust

The root of trust is a foundational component in hardware security, providing a secure anchor for the system's integrity. It is responsible for verifying the authenticity of hardware and software components during the boot process. This component is needed to always behave in the expected way, because its misbehavior cannot be detected by the system itself.

1.1.3 Trust Anchor

A trust anchor serves as a fundamental element in establishing trust within a system. Rather than relying on external validation, such as a certificate authority, a trust anchor is typically embedded directly into hardware or software, or provisioned securely through dedicated channels.

1.2 Trusted Platform Module (TPM)

The Trusted Platform Module (TPM) is a specialized hardware component, specifically a microcontroller, designed to enhance the security of computing devices. It can be a discrete chip on the motherboard or integrated into the CPU.

It provides a secure environment for generating, storing and managing cryptographic keys, ensuring that sensitive operations are performed in a protected manner. It allows to perform platform integrity measurements, and is used also for secure boot enforcement.

It acts as a harware-based root of trust, providing a secure foundation for establishing a chain of trust for the entire system during the boot process.

It defines three kinds of Roots of Trust, cooperating to measure, store, and report trusted information:

- **Root of Trust for Measurement (RTM)**: Responsible for making inherently reliable integrity measurements of the system's state during the boot process.
- **Root of Trust for Storage (RTS)**: Provides secure storage for cryptographic keys and maintains an accurate summary of values of integrity digests and the sequence of digests.
- **Root of Trust for Reporting (RTR)**: Facilitates the reliable reporting of the system's state and integrity measurements to remote entities.

In the figure 1.1, the architecture of a TPM is shown, showing the interaction between the different Roots of Trust components.

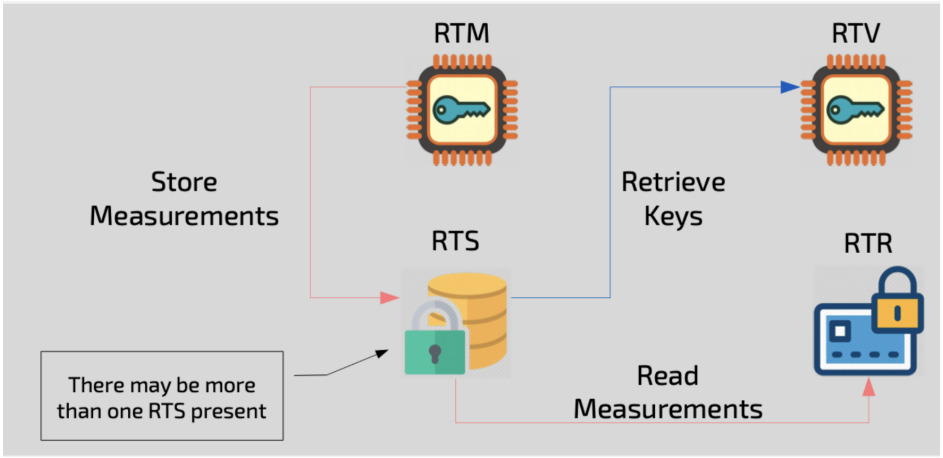

Figure 1.1: TPM Architecture and Roots of Trust

1.2.1 Certification and Attestation

Certification and attestation are essential for establishing and verifying trust in TPM-enabled platforms.

Certification uses cryptographic certificates to validate TPM components, starting with a key shipped with the TPM. This certified key forms the basis for a chain of trust, enabling secure storage and use of signing keys to prove system integrity.

Attestation allows the TPM to prove its state and authenticity to external entities, using unique identifiers (e.g., from Physical Unclonable Functions) and cryptographic evidence. Attestation covers compliance, the presence of trusted roots, and protection of key pairs.

Modern hierarchical attestation extends beyond one-time certification, enabling ongoing, layered verification of the platform's trustworthiness, including the TPM, boot process, and system configuration.

1.3 Tools for Implementation and Testing

To implement and test the TPM from the hardware side, we used QEMU.

QEMU is an open-source machine emulator and virtualizer that allows you to test and run harware platforms and operating systems in a virtualized environment. It supports a wide range of architectures

and can emulate various hardware components, including CPUs, memory, and peripherals. It is widely used for development, testing, and debugging purposes, providing a flexible and efficient way to simulate hardware systems. It is particularly useful for testing firmware and software in a controlled environment before deploying it on actual hardware.

In this case it was used to emulate the hardware platform, adding the TPM component to the emulated system.

CHAPTER 2

Implementation

The implementation of the Trusted Platform Module (TPM) involves the development of both hardware and firmware components.

The implementation of the Trusted Platform Module (TPM) requires the coordinated development of hardware and firmware components. Together, these components enable the execution of TPM commands and the management of cryptographic operations.

The implementation focuses on three core aspects:

- **Command Processing:** The system must be able to interpret and execute the TPM commands sent by the host system. This involves parsing the command structure, validating the parameters, and executing the corresponding operations.
- **Cryptographic Operations:** The system must implement the cryptographic algorithms required by the TPM, such as RSA, ECC, and SHA. These operations are essential for secure key generation, signing, and verification processes.
- **State Management:** The system must maintain the internal state of the TPM, including the storage of keys, policies, and other sensitive data. This involves managing the non-volatile memory and ensuring that the data is securely stored and protected against unauthorized access.

The TPM commands are the interface through which the TPM interacts with the host system. Each command is defined by a specific structure and set of parameters, as defined by the TPM specification of the communication protocol.

2.1 QEMU Implementation

The basic board is a custom ARM board developed to emulate a NXP board. The peripheral will be emulated along this board. The custom TPM was implemented as a standard peripheral device within QEMU, interfacing with the microcontroller via a memory-mapped bus. The module is designed to operate in parallel with the main system, simulating the behavior of a hardware TPM as closely as possible. This includes replicating the internal state transitions, initialization sequence, and command-response protocol typical of a real TPM device. Although integrated as a memory-mapped device, the TPM uses a serial-like communication protocol to receive and transmit data. To maintain the behavior of a real TPM. So a serial protocol was used. The device expects to receive data in a certain order, the same data and the same order as a real TPM and in the same way it does for the response. The peripheral adheres to a simplified model of the TPM 2.0 architecture.

2.1.1 Serial protocol

The TPM communicates using a byte-wise serial protocol. The firmware sends command packets one byte at a time, which the TPM receives and parses sequentially. Each command is reconstructed according to the TPM 2.0 command buffer format, using little-endian byte order for multi-byte fields. Each command must begin with a standard TPM header, which includes a tag, total size, and command code. Based on the command code, the TPM determines how to interpret the remaining payload, which varies depending on the specific command. Throughout the parsing process, the TPM validates the size of the command payload against the expected size. If there is a mismatch, an error code is returned immediately, halting further processing. The response is constructed in a similar manner: once command processing is complete, the TPM generates a response buffer with a corresponding header and payload. The firmware is expected to read the response only after the complete message has been sent by the TPM.

2.1.2 Implemented Functionalities

The core functionality of our TPM implementation centers around RSA key management and cryptographic operations, using the OpenSSL library to ensure correctness and cryptographic safety. The following TPM 2.0-style commands were implemented:

- **CreatePrimary**: Generates a new RSA key pair at the top of the TPM hierarchy. Internally, this is performed using OpenSSL's `EVP_PKEY` interface. The key is generated with a configurable size, given in input, encoded in DER format, and stored securely in the TPM's internal state.
- **RSA_Encrypt**: Accepts input data from the host and encrypts it using the public RSA key. The encryption uses PKCS#1 v1.5 padding and OpenSSL's high-level EVP API for secure buffer management.
- **RSA_Decrypt**: Uses the RSA private key stored in the TPM to decrypt ciphertext received from the host. Padding and integrity are validated automatically by OpenSSL.

To simulate a real-world TPM behavior, the following utility commands were also implemented:

- **Startup**: Initializes the TPM, resets its internal state, and prepares it for key generation and command processing.
- **SelfTest**: Performs a basic internal verification of key generation and encryption/decryption functionality.
- **Shutdown**: Transitions the TPM into an inactive state. On shutdown, all temporary key material is invalidated, and the TPM must be restarted before accepting new commands.

The RSA key generation process relies on OpenSSL's EVP API and supports dynamic key size configuration. The function `generate_rsa_keypair()` creates a new key pair and extracts both public and private key components in DER-encoded format.

All encryption and decryption operations are performed via the EVP interface using a secure context (`EVP_PKEY_CTX`). The code also includes detailed error reporting and memory safety checks to prevent buffer misuse.

Finally, we implemented an internal `qemu_verify_integrity()` function that validates the correctness of the RSA pipeline by encrypting and decrypting a test message. This self-test step ensures proper functioning of the OpenSSL-based cryptographic backend.

2.2 Firmware

The firmware implementation is crucial for the TPM's functionality, as it manages the interaction between the hardware and the software layers.

The firmware is responsible for executing the TPM commands, managing the cryptographic operations, and maintaining the internal state of the TPM. It is designed to be efficient and secure, ensuring that the TPM can perform its functions without exposing sensitive data or being vulnerable to attacks.

The firmware is implemented in C as a bare-metal application, meaning it runs directly on the hardware without an operating system.

It is structured into several files, each handling specific aspects of the TPM's functionality. The main components of the firmware include:

- **main.c:** The entry point of the firmware, which initializes the TPM and starts the command processing loop.
- **tpm.c (tpm.h):** Contains the core logic for processing TPM commands, including parsing the command structure, validating parameters, and executing the corresponding operations.
- **uart.c (uart.h):** Handles the UART communication between the TPM and the host system, ensuring that commands are received and responses are sent correctly.

2.2.1 main.c

The `main.c` file serves as the entry point for the TPM firmware. Its primary responsibilities are to initialize the TPM device, execute a sequence of TPM commands, and manage communication with the host system via UART.

The main steps performed in `main.c` are:

- **Initialization:** The UART interface is initialized for communication, and the TPM device structure is set up.
- **Startup and Self-Test:** The TPM is started and a self-test is performed to verify its integrity and functionality.
- **Key Management:** A primary key is created for cryptographic operations.
- **Cryptographic Operations:** The firmware demonstrates encryption and decryption using RSA, showing how data can be securely processed within the TPM.
- **Shutdown:** The TPM is properly shut down to ensure all sensitive operations are terminated securely.
- **Status Reporting:** Throughout the process, status messages are sent to the host via UART to indicate progress and results.

This structure ensures that the TPM firmware can reliably process commands, manage keys, and perform cryptographic operations while maintaining secure communication with the host system.

2.2.2 tpm.c

The `tpm.c` file contains the core logic for processing TPM commands and managing the TPM device state.

It provides functions for sending commands to the TPM, receiving responses, and handling cryptographic operations such as key creation, encryption, and decryption.

The main responsibilities of `tpm.c` are:

- **TPM Initialization:** Functions to initialize the TPM device and set its state, ensuring it is ready to process commands.
- **Command Communication:** Routines to send commands to the TPM and receive responses, with integrated UART logging for debugging and monitoring.
- **Specific Command Implementations:** For each TPM command (e.g., Startup, Self Test, Create Primary, RSA_Encrypt, RSA_Decrypt, Shutdown), dedicated functions are provided. These functions construct and initialize the corresponding command and response structures as defined in `tpm.h`, ensuring correct formatting and parameter handling for each operation.

The `tpm.c` file is essential for the TPM's functionality, as it encapsulates the logic for command processing and cryptographic operations while ensuring that the TPM adheres to the specifications defined in the TPM standard.

It is designed to be modular and extensible, allowing for future enhancements and additional commands to be easily integrated.

tpm.h

The `tpm.h` file defines the data structures and constants used in the TPM firmware. It serves as a header file that provides the necessary definitions for the TPM commands, response structures, and cryptographic parameters used throughout the firmware.

It includes definitions for:

- **Command Codes:** Constants representing the various TPM commands, such as `TPM2_CC_Startup`, `TPM2_CC_CreatePrimary`, and others.
- **Memory Regions:** Definitions for the base address and size of the TPM peripheral memory region, allowing for direct memory-mapped I/O access.
- **Startup Types:** Defines the startup modes for the TPM, such as `TPM_SU_CLEAR` and `TPM_SU_STATE`, which determine how the TPM initializes its state.
- **Structure Tags:** Specifies tags for TPM command and response structures, including `TPM_ST_NO_SESSION`, `TPM_ST_SESSION`, used to identify the type of TPM structure being processed.
- **Data Structures:** Definitions for the various data structures used in TPM commands and responses, including for example `tpm_createPrimary_command`, `tpm_createPrimary_response`, and others.
- **Response Codes:** Definitions for the response codes used by the TPM to indicate success or failure of operations.

This file is crucial for ensuring that the firmware can correctly interpret and construct the TPM command and response structures, as well as manage the cryptographic operations required by the TPM.

2.2.3 uart.c

The `uart.c` file implements the UART communication functions used by the TPM firmware to exchange data with the host system. Since the firmware runs on a bare-metal system without an operating system, a dedicated UART module is required to handle all serial communication directly with the hardware. It provides routines for initializing the UART interface, sending characters and strings, and printing data in hexadecimal format for debugging and status reporting purposes. The main functions in `uart.c` include:

- **UART_init:** Initializes the UART interface, setting the baud rate and enabling the UART controller.
- **UART_putstr:** Sends a null-terminated string over UART, character by character.
- **UART_putc:** Sends a single character over UART, waiting for the transmit buffer to be ready.
- **UART_println:** Sends a newline character to indicate the end of a line in the output.
- **UART_puthex:** Converts a 32-bit value to its hexadecimal representation and sends it over UART.
- **UART_puthex_byte:** Converts an 8-bit value to its hexadecimal representation and sends it over UART.
- **UART_puthex_2byte:** Converts a 16-bit value to its hexadecimal representation and sends it over UART.
- **UART_putdigit:** Sends a single digit (0-9) as a character over UART.

These functions ensure reliable communication between the TPM and the host, supporting both textual and hexadecimal output for monitoring and debugging purposes.

uart.h

The `uart.h` file provides the interface and definitions for UART communication used in the TPM firmware: it includes register definitions for the UART hardware and declares the functions implemented in `uart.c`.

This header ensures that the UART functions are accessible throughout the firmware, enabling reliable communication between the TPM and the host system.

CHAPTER 3

Limitations and Security Threats

Our custom TPM implementation does not guarantee complete security and presents several limitations inherent to its design and execution environment.

Communication Security

One of the primary limitations lies in the lack of secure communication between the TPM and the microcontroller. Operating within a QEMU environment, it is extremely challenging to implement a communication channel that is resistant to side-channel attacks. As a result, sensitive data transmitted to or from the TPM could potentially be intercepted or manipulated.

Key Storage

Another significant concern is secure key storage. Although keys are stored in a dedicated region of memory that is only accessible through the TPM API, this isolation is purely virtual. In a real TPM, keys would be stored in tamper-resistant non-volatile memory, physically protected from direct access. In our implementation, the lack of true hardware isolation leaves keys vulnerable to extraction in case of system compromise.

Lack of Command Validation

The TPM currently accepts and processes any well-formed command it receives, without enforcing higher-level policy or authentication. This makes it susceptible to misuse if malicious code running on the host sends unauthorized or crafted commands. Since our TPM focuses solely on cryptographic operations and does not implement access control or command authorization mechanisms, it cannot detect or prevent such abuse.

No Secure Boot

The implementation does not support secure boot or firmware integrity verification mechanisms. In a production TPM, secure boot ensures that only authenticated and untampered firmware is loaded, establishing a root of trust at startup. Without this feature, a compromised or malicious bootloader or kernel could gain unrestricted access to TPM services, undermining its security guarantees.

Lack of Logging

The TPM does not provide audit or activity logs for command execution, state transitions, or key usage. Such logging is vital in real-world scenarios for detecting misuse, tracing the origin of cryptographic operations, and supporting forensic analysis after a security incident. The absence of logs limits the system's accountability and makes it difficult to detect or respond to abnormal behavior.

Summary

In summary, while our TPM implementation is functionally representative and suitable for development and testing purposes, it lacks several critical hardware-level protections and secure communication features required for deployment in real-world, security-sensitive applications.

CHAPTER 4

Further Improvements

While our TPM implementation serves as a functional and educational prototype, several enhancements can be made to improve its robustness, realism, and security within the QEMU environment. These improvements aim to mitigate some of the identified threats and expand the TPM's capabilities while maintaining feasibility.

Secure Communication Channel

To reduce the risk of interception and tampering, a secure communication protocol could be introduced between the TPM and the microcontroller. For example, implementing an encrypted transport layer over the serial interface, using pre-shared keys or session-based encryption, would significantly enhance confidentiality and integrity.

Command Filtering and Validation

Introducing a command validation layer that verifies caller identity, and checks for malformed or unauthorized commands would help prevent misuse.

Secure Key Storage

In a QEMU-based environment, where true hardware-based isolation is not possible, secure key storage must be emulated through software mechanisms. In our implementation, the TPM generates a primary RSA key that serves as a trust anchor. This key acts as the root of trust for subsequent cryptographic operations, including the protection of other keys. To simulate secure storage, the primary key is used to encrypt newly generated keys before they are written to memory. Similarly, any key retrieved from memory must be decrypted using the primary key before it can be used. This approach ensures that even if the memory region is exposed, the protected keys remain unintelligible without access to the primary key. While this method does not provide the same level of tamper resistance as a physical TPM, it conceptually mirrors the role of a hardware-based Root of Trust for Storage (RTS), enabling integrity and confidentiality in key management within the emulated TPM. With the only effect to reduce the velocity of the module.

Basic Secure Boot Emulation

While full secure boot is not feasible in QEMU, a simplified version could be implemented where the TPM stores hashes of known-good firmware. During startup, the microcontroller could send a hash of the running firmware to the TPM for validation, simulating the boot integrity check process.

Logging Support

To increase transparency and accountability, a basic log feature could be added. The TPM could maintain an in-memory or file-backed log of all received commands, cryptographic operations, and internal state changes. These logs could be used for debugging or future security analysis.

Enhanced Error Handling and Feedback

Currently, the TPM returns only basic error codes for invalid commands. This can be extended to provide more detailed diagnostic feedback for malformed payloads, unsupported commands, or protocol violations. Such enhancements would make the system easier to debug and more resilient to unexpected inputs.

Support for Additional TPM Commands

To increase the realism of the implementation, support for additional TPM 2.0 commands could be developed, such as:

- **PCR Extend and Read:** Emulate platform configuration registers to support integrity measurements.
- **NV Storage:** Simulate non-volatile storage for keys, counters, and configuration values.

CHAPTER 5

References

The following references have been instrumental in the development of the TPM firmware and its associated documentation. These sources provide essential background information, technical specifications, implementation guidance, and best practices that have informed both the design and implementation of the system.

- **Trusted Platform Module (TPM) 2.0 Library Specification, Part 3 – Commands, Revision 1.38**
  `https://trustedcomputinggroup.org/wp-content/uploads/TPM-Rev-2.0-Part-3-Commands-01.38.pdf`
  This document specifies the command interface of the TPM 2.0, including command codes, structures, and expected behavior. It is a core reference for developing compliant TPM firmware.

- **tpm2-tss: TPM2.0 TSS (Trusted Software Stack)**
  `https://github.com/tpm2-software/tpm2-tss`
  The tpm2-tss project is an open-source implementation of the TPM 2.0 software stack. It includes the TCG TSS 2.0 system API (SAPI) and enhanced system API (ESAPI) layers. This repository served as a reference for software integration, testing, and compatibility.

- **Taming QEMU: Instrumenting and Modifying Emulated Systems**
  `https://github.com/quarkslab/sstic-tame-the-qemu`
  This project explores techniques for analyzing, instrumenting, and extending QEMU to emulate complex hardware environments such as TPMs. It provides insights into how to modify QEMU to support custom hardware features and debugging capabilities.